Date and Time Functions — Timestamps, Intervals, and Time Zones

> ***Chapter 17 | PostgreSQL Complete Guide*** *Module: 02_SQL_Basics | Difficulty: Beginner–Intermediate | Estimated Reading Time: 50 minutes*

Table of Contents

Learning Objectives

By the end of this chapter, you will be able to:

- Distinguish between DATE, TIME, TIMESTAMP, TIMESTAMPTZ, and INTERVAL types
- Retrieve and format the current date/time using NOW, CURRENT_DATE, CURRENT_TIMESTAMP
- Extract date parts with EXTRACT and DATE_PART, and truncate with DATE_TRUNC
- Perform date arithmetic with interval expressions and the AGE function
- Convert timestamps between time zones using AT TIME ZONE and understand UTC storage

---

Theory: Date and Time Types in PostgreSQL

Type Overview

Type	Storage	Description	Example
`DATE`	4 bytes	Calendar date only	`2024-06-15`

`TIME`	8 bytes	Time of day, no TZ	`14:30:00`
`TIME WITH TIME ZONE`	12 bytes	Time + UTC offset	`14:30:00+05:30`
`TIMESTAMP`	8 bytes	Date + time, no TZ	`2024-06-15 14:30:00`
`TIMESTAMPTZ`	8 bytes	Date + time, UTC-stored	`2024-06-15 14:30:00+00`
`INTERVAL`	16 bytes	Time span	`1 year 3 months 5 days`

TIMESTAMP vs TIMESTAMPTZ

This is the most important distinction:

- **TIMESTAMP** (without time zone): stores exactly what you give it. No time zone conversion. The database has no idea what time zone it represents.
- **TIMESTAMPTZ** (timestamp with time zone): stores in UTC internally. Converts to the session's `TimeZone` on output. **This is almost always what you want.**

```
SET timezone = 'US/Eastern';

INSERT INTO events (ts_no_tz, ts_with_tz) VALUES
    ('2024-06-15 10:00:00', '2024-06-15 10:00:00');

SET timezone = 'UTC';
SELECT ts_no_tz, ts_with_tz FROM events;
-- ts_no_tz:   2024-06-15 10:00:00     (unchanged)
-- ts_with_tz: 2024-06-15 14:00:00+00  (converted: EST is UTC-4)
```

INTERVAL

INTERVAL stores a duration:

```
INTERVAL '1 year'
INTERVAL '3 months 2 weeks 4 days 5 hours 30 minutes'
INTERVAL 'P1Y3M5DT4H30M'  -- ISO 8601 format
```

Getting the Current Date and Time

Function Reference

Function	Type Returned	Notes
`NOW()`	TIMESTAMPTZ	Current transaction start time
`CURRENT_TIMESTAMP`	TIMESTAMPTZ	Same as NOW()
`CURRENT_DATE`	DATE	Current date in session TZ
`CURRENT_TIME`	TIMETZ	Current time in session TZ
`CLOCK_TIMESTAMP()`	TIMESTAMPTZ	Actual current time (changes within transaction)

`TRANSACTION_TIMESTAMP()`	TIMESTAMPTZ	Same as NOW()
`STATEMENT_TIMESTAMP()`	TIMESTAMPTZ	Time this statement started
`TIMEOFDAY()`	TEXT	Current time as text string

Transaction vs Clock Time

```
BEGIN;
SELECT NOW();               -- 2024-06-15 10:00:00.000  (transaction start)
SELECT pg_sleep(2);
SELECT NOW();               -- 2024-06-15 10:00:00.000  (still the same!)
SELECT CLOCK_TIMESTAMP();   -- 2024-06-15 10:00:02.123  (actual current time)
COMMIT;
```

`NOW()` returns the **transaction start time** — it is constant within a transaction. Use `CLOCK_TIMESTAMP()` for elapsed time measurement.

```
-- Common current time queries
SELECT CURRENT_DATE;                        -- DATE: 2024-06-15
SELECT CURRENT_TIMESTAMP;                   -- TIMESTAMPTZ: 2024-06-15 10:00:00+00
SELECT CURRENT_TIMESTAMP(0);                -- truncated to seconds
SELECT NOW() AT TIME ZONE 'America/New_York'; -- in specific TZ
```

EXTRACT and DATE_PART

Both functions extract a component from a date/time value. EXTRACT is SQL standard; DATE_PART is the PostgreSQL function form (identical behavior).

```
EXTRACT(field FROM source)
DATE_PART('field', source)
```

Available Fields

Field	Description	Example (for 2024-06-15 14:30:45.123)
`year`	Year	2024
`month`	Month (1-12)	6
`day`	Day of month (1-31)	15
`hour`	Hour (0-23)	14
`minute`	Minute (0-59)	30
`second`	Seconds including fractional	45.123
`milliseconds`	Seconds * 1000	45123.0

`microseconds`	Seconds * 1000000	45123000.0
`dow`	Day of week (0=Sunday, 6=Saturday)	6
`doy`	Day of year (1-366)	167
`week`	ISO week number (1-53)	24
`quarter`	Quarter (1-4)	2
`epoch`	Seconds since 1970-01-01 00:00:00 UTC	1718458245.123
`timezone`	Offset from UTC in seconds	0
`timezone_hour`	Hour part of TZ offset	0

```
SELECT EXTRACT(YEAR  FROM CURRENT_DATE) AS year;        -- 2024
SELECT EXTRACT(MONTH FROM CURRENT_DATE) AS month;       -- 6
SELECT EXTRACT(DOW   FROM CURRENT_DATE) AS day_of_week; -- 0-6
SELECT EXTRACT(EPOCH FROM NOW())        AS unix_ts;     -- Unix timestamp

SELECT DATE_PART('year',  CURRENT_DATE) AS year;
SELECT DATE_PART('month', CURRENT_DATE) AS month;
```

EXTRACT vs date_part — Modern PostgreSQL (14+)

In PostgreSQL 14+, EXTRACT returns `NUMERIC`, while DATE_PART returns `DOUBLE PRECISION`. Use EXTRACT for precision in aggregations.

DATE_TRUNC — Truncating Timestamps

`DATE_TRUNC` truncates a timestamp to the specified precision, zeroing out smaller components.

```
DATE_TRUNC(field, source [, time_zone])
```

Truncation Levels

```
-- Source: 2024-06-15 14:37:52.123456

SELECT DATE_TRUNC('year',        '2024-06-15 14:37:52'::TIMESTAMP); -- 2024-01-01
00:00:00
SELECT DATE_TRUNC('quarter',     '2024-06-15 14:37:52'::TIMESTAMP); -- 2024-04-01
00:00:00
SELECT DATE_TRUNC('month',       '2024-06-15 14:37:52'::TIMESTAMP); -- 2024-06-01
00:00:00
SELECT DATE_TRUNC('week',        '2024-06-15 14:37:52'::TIMESTAMP); -- 2024-06-10
00:00:00 (Monday)
SELECT DATE_TRUNC('day',         '2024-06-15 14:37:52'::TIMESTAMP); -- 2024-06-15
00:00:00
SELECT DATE_TRUNC('hour',        '2024-06-15 14:37:52'::TIMESTAMP); -- 2024-06-15
```

```
14:00:00
SELECT DATE_TRUNC('minute',      '2024-06-15 14:37:52'::TIMESTAMP); -- 2024-06-15
14:37:00
SELECT DATE_TRUNC('second',      '2024-06-15 14:37:52.123'::TIMESTAMP); -- 2024-06-
15 14:37:52
SELECT DATE_TRUNC('milliseconds','2024-06-15 14:37:52.123456'::TIMESTAMP); --
...52.123
```

Grouping by Time Period

```
-- Monthly revenue report
SELECT DATE_TRUNC('month', order_date) AS month,
       SUM(total_amount)               AS monthly_revenue,
       COUNT(*)                        AS order_count
FROM   orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;

-- Hourly event counts
SELECT DATE_TRUNC('hour', event_time) AS hour_bucket,
       COUNT(*)                       AS events
FROM   access_logs
GROUP BY hour_bucket
ORDER BY hour_bucket;
```

AGE — Calculating Intervals Between Dates

```
AGE(timestamp)                  -- age from current date
AGE(timestamp1, timestamp2)     -- interval from timestamp2 to timestamp1
```

Returns an INTERVAL in years, months, days (human-readable format):

```
SELECT AGE('1990-05-20'::DATE);              -- 34 years 1 mon 26 days (approximate)
SELECT AGE('2024-06-15'::DATE, '1990-05-20'::DATE); -- 34 years 0 mons 26 days

-- Current age of employees
SELECT first_name, birth_date,
       AGE(birth_date)                         AS age_interval,
       EXTRACT(YEAR FROM AGE(birth_date))::INT AS age_years
FROM   employees;
```

Difference in Days vs AGE

```
-- Days between two dates (simple arithmetic)
SELECT '2024-06-15'::DATE - '2024-01-01'::DATE AS days_diff;   -- 166

-- Months between (approximate)
```

```
SELECT EXTRACT(YEAR  FROM AGE(end_date, start_date)) * 12 +
       EXTRACT(MONTH FROM AGE(end_date, start_date)) AS months
FROM   contracts;
```

MAKE_DATE and MAKE_TIMESTAMP

Build a date/timestamp from component integers:

```
MAKE_DATE(year, month, day)
MAKE_TIME(hour, minute, second)
MAKE_TIMESTAMP(year, month, day, hour, minute, second)
MAKE_TIMESTAMPTZ(year, month, day, hour, minute, second [, timezone])
MAKE_INTERVAL(years, months, weeks, days, hours, minutes, seconds)
```

```
SELECT MAKE_DATE(2024, 6, 15);           -- 2024-06-15
SELECT MAKE_TIME(14, 30, 0);             -- 14:30:00
SELECT MAKE_TIMESTAMP(2024, 6, 15, 14, 30, 0); -- 2024-06-15 14:30:00

-- Useful for constructing dates from columns
SELECT MAKE_DATE(year_col, month_col, 1) AS first_of_month
FROM   budget_entries;

-- Interval from parts
SELECT MAKE_INTERVAL(years => 1, months => 6, days => 15);
-- 1 year 6 mons 15 days
```

TO_CHAR — Formatting Dates as Strings

```
TO_CHAR(value, format_string)
```

Common Format Codes

Code	Description	Example
`YYYY`	4-digit year	2024
`YY`	2-digit year	24
`MM`	Month number, zero-padded	06
`Month`	Full month name	June
`Mon`	Abbreviated month	Jun
`DD`	Day of month, zero-padded	15
`Day`	Full day name	Saturday
`Dy`	Abbreviated day	Sat

`D`	Day of week (1=Sunday)	7
`HH24`	Hour (00-23)	14
`HH12` / `HH`	Hour (01-12)	02
`MI`	Minute	30
`SS`	Second	45
`MS`	Millisecond	123
`AM` / `PM`	Meridiem indicator	PM
`TZ`	Time zone abbreviation	UTC
`TZH:TZM`	Time zone offset	-05:00
`Q`	Quarter	2
`IW`	ISO week number	24
`WW`	Week of year	24

```
SELECT TO_CHAR(NOW(), 'YYYY-MM-DD');                    -- '2024-06-15'
SELECT TO_CHAR(NOW(), 'DD/MM/YYYY');                    -- '15/06/2024'
SELECT TO_CHAR(NOW(), 'Month DD, YYYY');                -- 'June      15, 2024'
SELECT TO_CHAR(NOW(), 'FMMonth DD, YYYY');              -- 'June 15, 2024' (FM
strips padding)
SELECT TO_CHAR(NOW(), 'YYYY-MM-DD HH24:MI:SS');         -- '2024-06-15 14:30:45'
SELECT TO_CHAR(NOW(), 'Day, DD Mon YYYY at HH12:MI AM'); -- 'Saturday, 15 Jun 2024
at 02:30 PM'
SELECT TO_CHAR(order_date, 'Q') AS quarter FROM orders; -- '2'
```

TO_DATE and TO_TIMESTAMP — Parsing Strings

```
TO_DATE(text, format_string)        -- returns DATE
TO_TIMESTAMP(text, format_string)   -- returns TIMESTAMPTZ
```

```
SELECT TO_DATE('15/06/2024', 'DD/MM/YYYY');              -- 2024-06-15
SELECT TO_DATE('June 15, 2024', 'FMMonth DD, YYYY');     -- 2024-06-15
SELECT TO_DATE('2024164', 'YYYYDDD');                    -- 2024-06-12 (day of year)

SELECT TO_TIMESTAMP('2024-06-15 14:30:45', 'YYYY-MM-DD HH24:MI:SS');
-- 2024-06-15 14:30:45+00

SELECT TO_TIMESTAMP('15-Jun-2024 02:30 PM', 'DD-Mon-YYYY HH12:MI AM');
```

Interval Arithmetic

Dates and timestamps support arithmetic with integers and intervals.

Date Arithmetic

```
-- Add/subtract days from DATE
SELECT CURRENT_DATE + 7;          -- 7 days from now
SELECT CURRENT_DATE - 30;         -- 30 days ago
SELECT '2024-06-15'::DATE + 100;  -- 100 days later

-- Subtract dates → integer (days)
SELECT '2024-12-31'::DATE - '2024-01-01'::DATE;  -- 365
```

Timestamp + Interval

```
-- Add an interval to a timestamp
SELECT NOW() + INTERVAL '1 hour';
SELECT NOW() + INTERVAL '3 months';
SELECT NOW() + INTERVAL '1 year 6 months 15 days';
SELECT NOW() - INTERVAL '2 hours 30 minutes';

-- Interval multiplication
SELECT INTERVAL '1 day' * 7;      -- 7 days
SELECT INTERVAL '1 hour' * 2.5;   -- 2:30:00

-- Interval addition
SELECT INTERVAL '1 month' + INTERVAL '15 days'; -- 1 mon 15 days
```

INTERVAL Functions

```
-- JUSTIFY_DAYS: convert excessive days to months
SELECT JUSTIFY_DAYS(INTERVAL '40 days');     -- 1 mon 10 days

-- JUSTIFY_HOURS: convert excessive hours to days
SELECT JUSTIFY_HOURS(INTERVAL '30 hours');   -- 1 day 06:00:00

-- JUSTIFY_INTERVAL: both
SELECT JUSTIFY_INTERVAL(INTERVAL '30 days 25 hours');  -- 1 mon 1 day 01:00:00
```

Business Calendar Calculations

```
-- Next Monday
SELECT DATE_TRUNC('week', CURRENT_DATE) + INTERVAL '7 days' AS next_monday;

-- First day of current month
SELECT DATE_TRUNC('month', CURRENT_DATE) AS first_of_month;

-- Last day of current month
SELECT (DATE_TRUNC('month', CURRENT_DATE) + INTERVAL '1 month' - INTERVAL '1
```

```
day')::DATE AS last_of_month;

-- Same day last year
SELECT CURRENT_DATE - INTERVAL '1 year';

-- Age in completed months
SELECT EXTRACT(YEAR FROM AGE(hire_date)) * 12 +
       EXTRACT(MONTH FROM AGE(hire_date)) AS months_tenure
FROM employees;
```

AT TIME ZONE

Converts a timestamp between time zones.

```
timestamp AT TIME ZONE zone    -- returns TIMESTAMPTZ if input is TIMESTAMP
                               -- returns TIMESTAMP if input is TIMESTAMPTZ
```

```
-- Convert UTC timestamp to US/Eastern
SELECT '2024-06-15 18:00:00'::TIMESTAMPTZ AT TIME ZONE 'America/New_York';
-- Returns: 2024-06-15 14:00:00  (UTC-4 in summer)

-- Convert US/Eastern local time to UTC
SELECT '2024-06-15 14:00:00'::TIMESTAMP AT TIME ZONE 'America/New_York';
-- Returns: 2024-06-15 18:00:00+00

-- Display NOW() in multiple time zones
SELECT NOW() AT TIME ZONE 'UTC'              AS utc,
       NOW() AT TIME ZONE 'America/New_York' AS new_york,
       NOW() AT TIME ZONE 'Asia/Kolkata'     AS india,
       NOW() AT TIME ZONE 'Europe/London'    AS london;
```

Session Time Zone

```
-- Show current session time zone
SHOW timezone;

-- Set session time zone
SET timezone = 'America/Chicago';
SET timezone = 'UTC';

-- Set at connection level (in postgresql.conf or per-database):
ALTER DATABASE mydb SET timezone = 'UTC';
```

ASCII Visual Diagrams

TIMESTAMP vs TIMESTAMPTZ Storage

```
User input:  '2024-06-15 10:00:00'  (session TZ = America/New_York, UTC-4)

TIMESTAMP (no TZ):                    TIMESTAMPTZ:
+---------------------------+          +---------------------------+
| Stored: 2024-06-15 10:00:00|         | Stored: 2024-06-15 14:00:00| <-- UTC
+---------------------------+          +---------------------------+
| Retrieval (any TZ): same   |         | Retrieval (EST): 10:00:00  |
|   2024-06-15 10:00:00      |         | Retrieval (UTC): 14:00:00  |
+---------------------------+          | Retrieval (IST): 19:30:00  |
                                       +---------------------------+
```

DATE_TRUNC Levels

```
Source: 2024-06-15 14:37:52.123456

TRUNC to:    Result:
-----------  --------------------------
year         2024-01-01 00:00:00
quarter      2024-04-01 00:00:00
month        2024-06-01 00:00:00
week         2024-06-10 00:00:00  (Monday of the week)
day          2024-06-15 00:00:00
hour         2024-06-15 14:00:00
minute       2024-06-15 14:37:00
second       2024-06-15 14:37:52
milliseconds 2024-06-15 14:37:52.123
```

Interval Arithmetic

```
Base date: 2024-01-31

+ INTERVAL '1 month'  = 2024-02-29  (PostgreSQL clamps to last day of Feb)
+ INTERVAL '31 days'  = 2024-03-02  (31 exact days later)

These are DIFFERENT results! Use INTERVAL '1 month' for calendar months,
use integer days for exact day counts.
```

SQL Examples

```sql
-- Example 1: Current date and time functions
SELECT CURRENT_DATE            AS today,
       CURRENT_TIME            AS now_time,
       CURRENT_TIMESTAMP       AS now_ts,
       NOW()                   AS now_alias,
       CLOCK_TIMESTAMP()       AS real_now;

-- Example 2: EXTRACT year, month, day
SELECT order_id,
       order_date,
```

```sql
       EXTRACT(YEAR  FROM order_date) AS order_year,
       EXTRACT(MONTH FROM order_date) AS order_month,
       EXTRACT(DAY   FROM order_date) AS order_day
FROM   orders;

-- Example 3: EXTRACT day of week
SELECT order_id,
       TO_CHAR(order_date, 'Day') AS day_name,
       EXTRACT(DOW FROM order_date) AS dow_number  -- 0=Sun, 6=Sat
FROM   orders;

-- Example 4: DATE_TRUNC for monthly grouping
SELECT DATE_TRUNC('month', order_date) AS month,
       COUNT(*)                        AS order_count,
       SUM(total_amount)               AS revenue
FROM   orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;

-- Example 5: DATE_TRUNC for weekly report
SELECT DATE_TRUNC('week', event_time) AS week_start,
       COUNT(*)                       AS event_count
FROM   system_events
GROUP BY week_start
ORDER BY week_start;

-- Example 6: AGE function
SELECT first_name, hire_date,
       AGE(hire_date) AS tenure
FROM   employees;

-- Example 7: AGE to get integer years
SELECT first_name,
       EXTRACT(YEAR FROM AGE(birth_date))::INT AS age_years
FROM   employees;

-- Example 8: Date arithmetic — add days
SELECT order_date,
       order_date + 30                   AS due_date,
       order_date + INTERVAL '45 days'   AS due_interval
FROM   orders;

-- Example 9: Date arithmetic — find overdue
SELECT order_id, order_date, shipped_date,
       shipped_date - order_date AS days_to_ship
FROM   orders
WHERE  shipped_date IS NOT NULL;

-- Example 10: INTERVAL arithmetic
SELECT NOW() + INTERVAL '1 year' AS one_year_from_now,
       NOW() - INTERVAL '6 months' AS six_months_ago,
       NOW() + INTERVAL '2 weeks 3 days 4 hours' AS complex_future;
```

```sql
-- Example 11: First and last day of month
SELECT DATE_TRUNC('month', CURRENT_DATE) AS first_day,
       (DATE_TRUNC('month', CURRENT_DATE) + INTERVAL '1 month - 1 day')::DATE AS
last_day;

-- Example 12: TO_CHAR for report formatting
SELECT employee_id,
       TO_CHAR(hire_date, 'FMMonth DD, YYYY') AS hire_date_formatted,
       TO_CHAR(salary, 'FM$999,999.00')       AS salary_formatted
FROM   employees;

-- Example 13: TO_DATE parsing European format
SELECT TO_DATE('15/06/2024', 'DD/MM/YYYY') AS parsed_date;

-- Example 14: TO_TIMESTAMP parsing with time
SELECT TO_TIMESTAMP('2024-Jun-15 14:30', 'YYYY-Mon-DD HH24:MI') AS parsed_ts;

-- Example 15: AT TIME ZONE conversion
SELECT NOW()                                  AS utc_time,
       NOW() AT TIME ZONE 'America/Los_Angeles' AS la_time,
       NOW() AT TIME ZONE 'Asia/Tokyo'          AS tokyo_time;

-- Example 16: MAKE_DATE from components
SELECT MAKE_DATE(2024, EXTRACT(MONTH FROM CURRENT_DATE)::INT, 1) AS
first_of_current_month;

-- Example 17: Quarter start date
SELECT DATE_TRUNC('quarter', CURRENT_DATE) AS quarter_start;

-- Example 18: Unix timestamp to TIMESTAMPTZ
SELECT TO_TIMESTAMP(1718458245) AS from_unix;

-- Example 19: TIMESTAMPTZ to Unix epoch
SELECT EXTRACT(EPOCH FROM NOW())::BIGINT AS unix_now;

-- Example 20: Reporting window — last 90 days
SELECT COUNT(*) AS orders_last_90_days
FROM   orders
WHERE  order_date >= CURRENT_DATE - INTERVAL '90 days';
```

Common Mistakes

1. **Using TIMESTAMP instead of TIMESTAMPTZ.** Without time zone awareness, timestamps are ambiguous — a timestamp stored during DST and retrieved outside DST will appear to have shifted. Always use TIMESTAMPTZ for events.

2. **Adding months with integer arithmetic.** `'2024-01-31'::DATE + 30` is 30 days later (March 1), not one month later. Use `+ INTERVAL '1 month'` for calendar-month arithmetic, which correctly gives February 29, 2024.

3. **EXTRACT(DOW ...) numbering.** In PostgreSQL, DOW goes 0=Sunday through 6=Saturday. ISODOW uses 1=Monday through 7=Sunday (ISO 8601). Know which you need.
4. **NOW() is constant within a transaction.** Using NOW() to measure elapsed time within a transaction always returns the same value. Use CLOCK_TIMESTAMP() for wall-clock time measurement.
5. **DATE_TRUNC('week') truncates to Monday.** In PostgreSQL, `DATE_TRUNC('week', date)` returns the Monday of that week (ISO week starts on Monday). If you need Sunday, add `- INTERVAL '1 day'` after truncating to week, then adjust.
6. **Forgetting FM in TO_CHAR.** `TO_CHAR(date, 'Month')` pads the month name to 9 characters: 'June '. Use `FM` prefix — `'FMMonth'` — to suppress padding.
7. **Date string ISO parsing.** PostgreSQL parses `'2024-06-15'::DATE` correctly as ISO 8601. But `'15-06-2024'::DATE` fails. Always use ISO 8601 format (YYYY-MM-DD) for date literals, or use TO_DATE with explicit format.

Best Practices

1. **Always store timestamps as TIMESTAMPTZ** — it stores in UTC and handles time zone conversion automatically. Only use TIMESTAMP without time zone for calendar/scheduling data where the time zone is meaningless (e.g., "9 AM every Tuesday regardless of DST").
2. **Store all times in UTC on the server** — set `timezone = 'UTC'` in postgresql.conf. Let the application layer handle display time zone conversion.
3. **Use INTERVAL for date arithmetic, not integer days** — `+ INTERVAL '1 month'` is calendar-correct; `+ 30` is not.
4. **Use DATE_TRUNC for period grouping in GROUP BY** — it creates stable, consistent bucket boundaries that work correctly across DST transitions.
5. **Index TIMESTAMPTZ columns** — date range queries (`WHERE created_at >= ... AND created_at < ...`) are among the most common; always index these columns.
6. **Use EXTRACT(EPOCH FROM ...) for cross-database duration calculations** — it gives absolute seconds, which is portable and avoids interval comparison complications.
7. **Prefer named time zones over UTC offsets** — `'America/New_York'` handles DST automatically; `'-05:00'` does not and will be wrong for half the year.

Performance Considerations

Indexes and Date Filtering

```
-- B-tree index on TIMESTAMPTZ — supports range queries
CREATE INDEX idx_orders_date ON orders (order_date);

-- Query using the index (range scan):
```

```sql
SELECT * FROM orders
WHERE  order_date >= '2024-01-01' AND order_date < '2025-01-01';
```

Avoid Functions on Indexed Columns in WHERE

```sql
-- SLOW: function prevents index on order_date
WHERE EXTRACT(YEAR FROM order_date) = 2024

-- FAST: range comparison uses index
WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01'
```

DATE_TRUNC in GROUP BY

DATE_TRUNC in GROUP BY does not prevent index use for the WHERE filter — only when it appears in the WHERE clause itself:

```sql
-- Fine: WHERE uses index; GROUP BY adds hash-agg step separately
SELECT DATE_TRUNC('month', order_date), SUM(total)
FROM   orders
WHERE  order_date >= '2023-01-01'   -- index used here
GROUP BY DATE_TRUNC('month', order_date);
```

Partitioning by Date

For very large time-series tables, consider range partitioning by date:

```sql
CREATE TABLE orders (order_id BIGINT, order_date DATE, ...)
PARTITION BY RANGE (order_date);

CREATE TABLE orders_2024 PARTITION OF orders
FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

Interview Questions

1. What is the difference between TIMESTAMP and TIMESTAMPTZ in PostgreSQL?
2. What does NOW() return, and how does it differ from CLOCK_TIMESTAMP()?
3. How does DATE_TRUNC('month', ts) work, and what does it return?
4. What is the difference between `date + 30` and `date + INTERVAL '1 month'`?
5. How does EXTRACT(DOW FROM date) number the days of the week?
6. How would you convert a TIMESTAMPTZ from UTC to a specific time zone for display?
7. What does AGE(timestamp) return, and what type is it?
8. How do you efficiently filter rows within the current calendar year?
9. What does `TO_CHAR(date, 'FMMonth')` do differently from `TO_CHAR(date, 'Month')`?
10. How would you calculate the number of complete months between two dates?

Interview Answers

Q1: TIMESTAMP vs TIMESTAMPTZ? TIMESTAMP stores date+time with no time zone context — it is a "naive" timestamp that means whatever you stored. TIMESTAMPTZ stores the value internally as UTC and automatically converts to the session's time zone on display. For any real-world event tracking, use TIMESTAMPTZ. TIMESTAMP is only appropriate for "absolute calendar" situations.

Q2: NOW() vs CLOCK_TIMESTAMP()? NOW() returns the transaction start time — it is constant throughout the entire transaction (every call to NOW() within one transaction returns the same value). CLOCK_TIMESTAMP() returns the actual wall-clock time at the moment of the call, changing between calls. Use CLOCK_TIMESTAMP() for measuring elapsed time.

Q3: DATE_TRUNC('month', ts)? Returns a TIMESTAMP/TIMESTAMPTZ with the month and year preserved but day set to 1, and time set to 00:00:00. Example: `DATE_TRUNC('month', '2024-06-15 14:30')` returns `2024-06-01 00:00:00`. Used to group data by month.

Q4: `date + 30` vs `date + INTERVAL '1 month'`? `date + 30` adds exactly 30 days (integer). `date + INTERVAL '1 month'` adds one calendar month, which is 28, 29, 30, or 31 days depending on the month. Example: `'2024-01-31' + INTERVAL '1 month'` = `2024-02-29` (PostgreSQL clamps to last day); `'2024-01-31' + 30` = `2024-03-01`.

Q5: DOW numbering? `EXTRACT(DOW FROM date)` returns 0=Sunday, 1=Monday, ..., 6=Saturday. `EXTRACT(ISODOW FROM date)` returns 1=Monday, ..., 7=Sunday (ISO 8601 standard). Use ISODOW for ISO week-based calculations.

Q6: Convert TIMESTAMPTZ for display? Use `AT TIME ZONE 'zone_name'`: `SELECT NOW() AT TIME ZONE 'America/Chicago'`. Or set the session time zone: `SET timezone = 'America/Chicago'` and all TIMESTAMPTZ output is automatically converted.

Q7: What AGE returns? AGE returns an INTERVAL. `AGE('1990-05-20')` returns something like `33 years 11 mons 15 days`. To get integer years, use `EXTRACT(YEAR FROM AGE(birth_date))::INT`.

Q8: Filter current calendar year?

```
WHERE order_date >= DATE_TRUNC('year', CURRENT_DATE)
  AND order_date <  DATE_TRUNC('year', CURRENT_DATE) + INTERVAL '1 year'
```

This is index-friendly. Avoid `EXTRACT(YEAR FROM order_date) = 2024` which prevents index use.

Q9: FMMonth vs Month? `Month` pads the month name to 9 characters: 'June ' (with trailing spaces). `FMMonth` (FM = Fill Mode) suppresses padding: 'June'. Always use FM prefix when you want clean, unpadded output.

Q10: Complete months between dates?

```
EXTRACT(YEAR FROM AGE(end_date, start_date)) * 12 +
EXTRACT(MONTH FROM AGE(end_date, start_date))
```

AGE computes the interval, then EXTRACT pulls year and month components and combines them.

Hands-on Exercises

Setup:

```sql
CREATE TABLE events (
    event_id    SERIAL PRIMARY KEY,
    event_name  TEXT NOT NULL,
    started_at  TIMESTAMPTZ NOT NULL,
    ended_at    TIMESTAMPTZ,
    created_by  TEXT
);

INSERT INTO events (event_name, started_at, ended_at, created_by) VALUES
('Conference A',  '2024-01-15 09:00:00+00', '2024-01-15 17:00:00+00', 'alice'),
('Workshop B',    '2024-02-20 13:00:00+00', '2024-02-20 16:30:00+00', 'bob'),
('Webinar C',     '2024-03-05 18:00:00+00', '2024-03-05 19:30:00+00', 'alice'),
('Summit D',      '2024-06-10 08:00:00+00', NULL,                      'carol'),
('Hackathon E',   '2024-11-01 09:00:00+00', '2024-11-03 18:00:00+00', 'bob'),
('Meetup F',      '2024-12-15 19:00:00+00', '2024-12-15 21:00:00+00', 'alice');
```

Exercise 1: Extract the year, month name, day of week name, and hour from started_at for each event.

Exercise 2: Calculate the duration of each event (ended_at - started_at) in hours. Show NULL for events with no end time.

Exercise 3: Group events by month (using DATE_TRUNC) and show total event count per month.

Exercise 4: Find all events that started in Q1 (January–March) of 2024.

Exercise 5: For events with both start and end times, calculate how many days ago each event ended (from today), and format started_at as 'DD Mon YYYY HH24:MI TZ'.

Solutions

```sql
-- Exercise 1
SELECT event_name,
       EXTRACT(YEAR  FROM started_at)         AS event_year,
       TO_CHAR(started_at, 'FMMonth')          AS month_name,
       TO_CHAR(started_at, 'FMDay')            AS day_of_week,
       EXTRACT(HOUR  FROM started_at)          AS start_hour
FROM   events;

-- Exercise 2
SELECT event_name,
       started_at,
       ended_at,
       ROUND(EXTRACT(EPOCH FROM (ended_at - started_at)) / 3600.0, 2) AS
duration_hours
FROM   events;

-- Exercise 3
SELECT DATE_TRUNC('month', started_at) AS event_month,
       COUNT(*)                        AS event_count
FROM   events
GROUP BY DATE_TRUNC('month', started_at)
```

```sql
ORDER BY event_month;

-- Exercise 4
SELECT event_name, started_at
FROM   events
WHERE  started_at >= '2024-01-01'
  AND  started_at <  '2024-04-01';

-- Exercise 5
SELECT event_name,
       TO_CHAR(started_at AT TIME ZONE 'UTC', 'DD Mon YYYY HH24:MI TZ') AS
start_label,
       (CURRENT_DATE - ended_at::DATE) AS days_ago
FROM   events
WHERE  ended_at IS NOT NULL
ORDER BY ended_at DESC;
```

Advanced Notes

`tstzrange` — Timestamp Ranges

PostgreSQL has a native range type for timestamp intervals:

```sql
-- Store event periods as ranges
CREATE TABLE bookings (
    room_id   INT,
    period    TSTZRANGE,
    EXCLUDE USING gist (room_id WITH =, period WITH &&)  -- prevent overlaps
);

SELECT * FROM bookings
WHERE period @> NOW();  -- currently active bookings
```

`pg_timezone_names` Catalog

```sql
-- List all available time zone names
SELECT name, abbrev, utc_offset, is_dst
FROM   pg_timezone_names
ORDER BY name;
```

Epoch Conversion

```sql
-- UNIX timestamp to TIMESTAMPTZ
SELECT TO_TIMESTAMP(1718458245) AS from_unix;

-- TIMESTAMPTZ to UNIX timestamp
SELECT EXTRACT(EPOCH FROM NOW())::BIGINT AS unix_ts;
```

ISO Week Calendar

ISO weeks start on Monday. Week 1 is the week containing the first Thursday of the year:

```
SELECT EXTRACT(ISOYEAR FROM date) AS iso_year,  -- may differ from calendar year for
week 52/1
       EXTRACT(WEEK    FROM date) AS iso_week,
       EXTRACT(ISODOW  FROM date) AS iso_dow    -- 1=Mon, 7=Sun
FROM (VALUES ('2024-12-31'::DATE)) AS t(date);
-- Dec 31 2024 is in ISO year 2025, week 1
```

Cross-References

- **Previous:** 07_numeric_functions.md — Numeric functions and casting
- **Next:** `03_Intermediate_SQL/01_joins.md` — JOINs with date-range conditions
- **Related:** 05_null_handling.md — NULL in date columns
- **Related:** 03_select_basics.md — ORDER BY with date columns, NULLS FIRST/LAST
- **Related (03_Intermediate):** `03_Intermediate_SQL/02_aggregations.md` — GROUP BY with DATE_TRUNC
- **Related (03_Intermediate):** `03_Intermediate_SQL/08_partitioning.md` — Range partitioning by date